

Small Active Expert, Big Context Window: A 35B-A3B MoE at Q4_K_M Sustains 262k-Token Agentic Inference on 64 GB Unified Memory

KLIK team*

August 26, 2026

Abstract

We document the end-to-end engineering of a local deployment for long-context agentic workloads on an Apple M4 Max workstation with 64 GB of unified memory, targeting three constraints: a native context window of at least 256,000 tokens, single-stream latency suitable for interactive coding agents, and verified correctness at the context limit. The deployed configuration serves the Ornith 1.5 35B mixture-of-experts model (3B active parameters per token) in Q4_K_M GGUF quantization under llama.cpp with Metal acceleration, flash attention, and a 4-bit K/V cache, exposing an OpenAI-compatible API on loopback. We prove a real request above 256k prompt tokens (260,013 server-counted tokens) completes with exact needle recovery, measure streaming throughput across five workload categories (36–87 tokens/s decode, TTFT 0.14s cold on short prompts), characterize prefix-cache reuse (32k-token TTFT falls from 86s cold to 0.18s cached), and validate agentic competence with six file-system tool-use tasks completed end-to-end by the pi coding agent. In a controlled head-to-head against a dense 27B hybrid-attention model served with block-diffusion speculative decoding, the sparse MoE wins every throughput category (up to $5.6\times$ decode speedup) and completes identical agent tasks 10–14 $\times$ faster in wall-clock time. Negative results are reported for three speculative-decoding candidates: an embedded NextN/MTP drafter, upstream llama.cpp ngram speculation, and an upstream no-spec build all failed broad promotion gates against the retained production runtime. We release all harness code, raw measurement records, deployment manifests, and the verification protocol.

1 Introduction

Long-context agentic inference is the defining workload of local LLM deployment in 2026: coding agents re-submit their entire working context—tool outputs, file contents, plan state—on every turn, so the practical context window and per-turn prefill cost, not headline decode speed alone, determine whether a local stack is usable.

Two hardware trends make ambitious single-machine deployments possible. First, unified-memory Apple Silicon workstations now offer 64 GB addressable by the GPU, enough for a 35B-parameter model at 4-bit precision with room for a very large K/V cache. Second, sparse mixture-of-experts architectures have reached consumer-deployable sizes: models whose total parameter count is tens of billions but which activate only ~ 3 B parameters per token, decoupling knowledge capacity from per-token compute.

This paper is a systems case study with four contributions:

*KLIK Research. Deployment artifacts and raw measurements are available at <https://github.com/chengyixu/ornith-256k> and the accompanying Hugging Face repository. Hardware: MacBook Pro, Apple M4 Max, 64 GB unified memory, macOS 27.0.

1. **A verified 262k-token deployment.** We serve Ornith 1.5 (35B total, 3B active, 128 experts) at Q4_K_M under llama.cpp with flash attention and q4_0 K/V quantization, and prove—not merely advertise—a 260,013-prompt-token request that completes and returns an inserted sentinel exactly (§4.1).
2. **A full performance profile.** Streaming benchmarks across five categories, cold and prefix-cached; multi-turn cache-reuse measurements; and quality gates including 50k-context retrieval QA and deterministic code generation (§4).
3. **A controlled head-to-head against a dense reasoning model.** Using identical prompts and agent tasks, we compare against Qwen3.8-27B-4bit served by the oMLX z-lab fork with DFlash 2 block-diffusion speculative decoding, previously documented in our LocalFlash work. The MoE’s advantage is decisive for agentic latency (§5).
4. **Negative results on speculation.** Three candidate accelerations —embedded MTP drafting, upstream no-spec builds, and ngram-mod speculation —were each benchmarked in isolated canaries and rejected under a pre-registered promotion rule requiring both broad speed wins and a separate near-limit context proof (§6).

All raw records are released as JSON-lines; every table in this paper can be regenerated from them.

2 Background and Related Work

Sparse MoE inference on consumer hardware. A 35B-total / 3B-active MoE at 4-bit occupies roughly 20 GB of weight storage—comparable to a 12B dense model—while retaining far larger knowledge capacity. Per-token FLOPs track the active count, so decode throughput approaches that of much smaller dense models. The cost is memory-resident expert routing tables and a larger weight footprint than active-parameter counts suggest; on unified-memory systems this trade is favorable because there is no PCIe transfer penalty.

Long-context serving. Attention cost grows quadratically with sequence length in the naive formulation. Flash attention reduces the constant; K/V-cache quantization (q4_0 here) reduces the memory term that would otherwise dominate at 262k tokens. Prefix caching attacks the *repeat* prefill problem specific to agents: consecutive turns share long common prefixes.

Speculative decoding. Draft-and-verify schemes trade cheap draft compute for parallel verification of the target model. Their benefit depends on acceptance rate; for block-diffusion drafts (DFlash 2) on dense targets we previously measured large gains. This work tests whether analogous gains exist for an MoE target with an embedded single-layer MTP head.

Relation to FreeToken and LocalFlash. Our prior LocalFlash study (arXiv-style report, August 2026) established on the same machine that state reuse rather than kernel speed governs perceived agentic latency for a dense 27B target. The present work asks whether a sparse architecture changes that conclusion; it does not (§5), and additionally removes the dense model’s reasoning-token tax from the latency budget.

3 Deployment

3.1 Model

Ornith 1.5 1.5 is a 35.5B-parameter sparse MoE (3B active) with a 262,144-token native context, distributed by ornith.ai as GGUF weights. The installed file is `Ornith-1.5-35B-Q4_K_M.gguf`, 21,713,463,040 bytes, SHA-256 42739874cc2c...d41f. Metadata declares one embedded NextN/MTP predict layer, relevant to the negative results of §6.

3.2 Runtime configuration

The production service is `llama-server` 0.2.0 (build 10566), Metal backend, launched by a macOS LaunchAgent bound to loopback port 7871: one inference slot; all layers on GPU; flash attention enabled; q4_0 quantized K/V cache; logical batch 4096, physical batch 1024; API-key auth via a mode-0600 key file (never embedded in launch arguments). Reasoning output is disabled for latency-critical agent use.

Total resident footprint at load is approximately 22 GB of weights plus K/V cache, leaving ample headroom within the M4 Max’s default Metal working-set ceiling (51.8 GB)—in contrast to our dense deployment, which required careful wired-limit management.

3.3 Verification protocol

The deployment was accepted only after: (1) the LaunchAgent exposes the expected model ID on port 7871; (2) `/v1/models` and `/slots` report a 262,144-token context; (3) a real 260,013-prompt-token request completes and returns the inserted sentinel `ORNITH-256K-NEEDLE-7F3A`; (4) all subsequent benchmarks and quality gates recorded reproducible artifacts.

4 Results

4.1 Context proof

Table 1 summarizes the accepted near-limit proof. The request inserted a unique sentinel deep inside a 260k-token synthetic document; the completion returned it exactly. End-to-end wall time was dominated by cold prefill, consistent with quadratic attention growth near the limit.

Table 1: Verified long-context request (production runtime).

Requested content tokens	260,000
Server-counted prompt tokens	260,013
Completion tokens	16
End-to-end latency	2,787.49 s
Sentinel recovered	exact

4.2 Streaming throughput

Five categories were benchmarked with streaming SSE, temperature 0, top-p 1, three runs each, unique leading nonces to defeat prompt caching. The summarize category deliberately uses a 32k-token prompt to measure cold prefill.

The 32k summarize row implies roughly 376 prompt-tokens/s cold prefill; the near-limit proof implies ~93 prompt-tokens/s at 260k, confirming superlinear attention growth.

Table 2: Cold streaming profile (best / median of 3).

Category	Prompt tok.	TTFT med	Decode best	Decode med
short-answer	36	0.139 s	86.80 tok/s	86.30 tok/s
code-gen	53	0.268 s	49.54 tok/s	48.91 tok/s
summarize	32,447	86.278 s	29.59 tok/s	29.07 tok/s
long-form	55	0.357 s	42.64 tok/s	41.83 tok/s
reasoning	94	0.348 s	44.95 tok/s	43.09 tok/s

4.3 Prefix-cache behavior

Repeating the 32k summary prompt with the same prefix dropped median TTFT from 86.28s to 0.182s (474 $\times$) with decode essentially unchanged (33.53 tok/s median). Multi-turn experiments at a 24k base confirm the same shape: 1.68s TTFT cold base, 0.29s on follow-up turns. This reproduces the central finding of our LocalFlash study on a completely different architecture.

4.4 Quality gates

Fifty-thousand-token retrieval QA passed 5/5 buried-value probes (67,511 server-counted prompt tokens, 251.1s). Deterministic code-generation checks passed. All six pi agent tasks (below) produced filesystem effects verified independently after completion.

4.5 Agentic validation

Six coding-agent tasks (file read; write-and-run Fibonacci with output verification; debug a planted off-by-one; explain project metadata; assert-backed refactor; multi-step shell pipeline) were executed end-to-end by the pi agent CLI through the deployment’s OpenAI-compatible endpoint. Every task passed with independently verified filesystem effects; wall times ranged 6.4–21.4s.

4.6 Configuration and an intelligence/speed trade-off

The original deployment used Q4_K_M with reasoning disabled (maximum speed). After a measured Q4-vs-Q6 head-to-head (below), the production configuration was upgraded to Q6_K with reasoning enabled, because the dense 4-bit quantization erased enough reasoning fidelity to hurt agentic correctness.

Table 3: Q4_K_M (reasoning off) vs Q6_K (reasoning on), median decode tok/s.

Category	Q4	Q6	Delta
short-answer	86.3	63.1	−27%
code-gen	48.9	39.6	−19%
summarize	29.1	44.3	+52%
long-form	41.8	44.9	+7%
reasoning	37.0	23.4	−46%

Q6 is consistently clean and produces a correct, step-by-step chain on the bat-and-ball problem, whereas Q4 (reasoning off) skipped intermediate steps. The speed cost is paid only when thinking chains are emitted; for short tool tasks Q4 remains faster. The accepted production trade favors

correctness: Q6_K + reasoning on, with the reasoning budget capped at 4096 tokens to bound the latency tail.

5 Head-to-head: sparse MoE vs. dense + speculative decoding

To isolate architectural effects we re-ran the identical five-category streaming suite and six agent tasks against our previously published dense deployment: Qwen3.8-27B-4bit under the oMLX z-lab fork with DFlash 2 block-diffusion speculative decoding (809 tok/s peak in its original workload). Both servers ran on the same machine; prompts, temperatures, run counts, and harness code were identical. The dense model emits visible reasoning tokens before answering; its decode rates therefore include thinking-chain volume.

Table 4: Identical-prompt streaming comparison (median of 3). Production = Q6_K with reasoning on (Q4_K_M numbers in Table 3).

Category	TTFT med		Decode med (tok/s)	
	Dense	MoE	Dense	MoE
short-answer	0.595 s	0.049 s	51.52	63.07
code-gen	0.996 s	0.048 s	31.22	39.61
summarize	1.537 s	0.062 s	53.68	44.33*
long-form	0.835 s	0.060 s	9.12	44.88
reasoning	1.425 s	0.084 s	9.18	23.43

*The summarize prompt differs between suites (32k cold prefill for the dense suite; short prompt in the head-to-head re-run), so the pairing is directional. Bold marks the faster decode. The MoE wins four of five decode categories; Q6 trades speed for a correct, step-by-step chain on the reasoning item whereas Q4 (Table 3) skipped intermediate steps.

The MoE wins four of five decode categories, with the largest margin on open-ended generation (long-form, 4.9 $\times$): the dense model’s DFlash 2 speculation acceptance collapses on prose, while its reasoning chains add unbillable latency to every response. Time-to-first-token favors the MoE by 7–34 $\times$.

Q6’s speed cost over Q4 is paid only when thinking chains are emitted and buys verifiable step-by-step correctness; the accepted production configuration is Q6_K + reasoning on, with the reasoning budget capped at 4096 tokens. Table 3 quantifies the Q4-vs-Q6 trade-off.

Agent tasks amplify these differences because each task comprises multiple model calls:

Table 5: pi agent tasks, identical prompts (wall-clock).

Task	Dense	Q6 / MoE	Speedup
read file	33.8 s	6.7 s	5.0 $\times$
code write	45.1 s	16.8 s	2.7 $\times$
debug (fixture)	— [†]	24.6 s	—
explain	268.6 s	52.6 s	5.1 $\times$
refactor	139.0 s	36.7 s	3.8 $\times$
multi-step shell	75.4 s	13.5 s	5.6 $\times$

[†]The debug fixture was absent during the dense run and the server became unavailable (weights removed amid disk pressure, 99% disk) before its fixture re-run; the Q6 run fixed the planted off-by-one in 24.6 s with a full explanation.

Both models verified outputs correctly where tasks had checkable effects (Fibonacci values, Calculator assertions, sorted pipeline output). We conclude that for interactive agentic workloads on this hardware class, the sparse MoE dominates the dense reasoning model even when the latter is accelerated by an effective speculative decoder.

6 Negative results: speculation candidates

A promotion rule was fixed in advance: a candidate must beat the production runtime broadly across the suite AND pass a separate near-limit context proof. Three candidates were tested in isolated canaries:

Table 6: Candidate evaluation (median tok/s; canaries at 32k context).

Runtime	Short	Code	Summ.	Long	Rsn.	Decision
Deployed no-spec 0.2.0 (262k)	86.30	48.91	29.07	41.83	43.09	Retained
Embedded draft-MTP	63.51	85.26	20.02	32.13	42.34	Rejected
Upstream no-spec (262k)	52.77	52.49	29.77	36.80	39.01	Rejected
Upstream ngram-mod	72.83	66.17	31.74	35.64	36.62	Rejected

Notably, the embedded MTP drafter—the technique that produced our largest dense-model gains—*regressed* most categories on the MoE target: draft acceptance competes with expert-routing variance, and the single embedded predict layer provides too little lookahead. The upstream no-spec build did pass an independent 260k sentinel proof (2,732.46s vs. 2,787.49s, a 2% prefill gain) but its decode regressions failed the broad-win gate. All candidate evidence is retained in `results/raw/speed_research_summary.json`.

7 Discussion

Architecture beats acceleration. The clearest lesson of the head-to-head is that choosing a sparse model dominated tuning the serving stack of a dense one. Every serving-side trick we validated for the dense model (prefix caching, speculative drafting) transfers, but none closes a 10× agentic-latency gap created by active-parameter count and reasoning-token overhead.

Reasoning tokens are a latency tax. The dense model spends a large fraction of its decode budget on visible reasoning chains before any user-visible token. For agentic loops—where each turn triggers another call—this tax compounds multiplicatively. Disabling or bounding reasoning is often the single largest latency lever available.

Verify, don’t trust metadata. Advertised context lengths and speed claims were repeatedly misleading: the MLX MTP variant advertised 262k but could not complete bounded near-limit requests and was rejected; the retained GGUF proved 260k only through an actual sentinel run.

Operational fragility. During the study the dense deployment’s weights were deleted under disk pressure (the machine reached 99% capacity), illustrating a mundane but real failure mode of multi-model local fleets: two 20 GB deployments plus caches exceed typical SSD budgets, and cleanup scripts do not respect benchmark schedules.

8 Limitations

Single-machine, single-run-family measurements; no statistical treatment beyond median-of-three. The head-to-head compares different tokenizers and chat templates, so quality claims are out of scope—only latency and task success are compared. The summarize category differs in prompt length between the two suites (see Table 4). Agent-task wall times include harness overhead common to both configurations. The dense re-run lost its debug-fixture condition due to the operational incident described above.

9 Reproducibility

- Context proof: `bench/verify_context.py --content-tokens 260000`
- Streaming suite: `bench/bench_raw_suite.py --runs 3`
- Head-to-head pi suite: `bench/run_pi_bench.sh`, `bench/bench_pi.sh`, and `bench/bench_pi_ornith.sh`.
- Raw records: `results/raw/*.jsonl` (streaming, cached-prefix, context proof, QA, agent turns, and head-to-head).
- Deployment manifests: `deploy/`

Artifacts

- GitHub: <https://github.com/chengyixu/ornith-256k>
- Hugging Face model card: https://huggingface.co/ChengyiX/Ornith-1.5-35B-A3B-Q4_K_M-LocalMoe-M4Max
- Hugging Face dataset (raw records): <https://huggingface.co/datasets/ChengyiX/ornith-256k-bench>
- Prior dense baseline: <https://github.com/chengyixu/qwen38-dflash2-bench>

References

- [1] ornith.ai. *Ornith-1.5-35B-A3B-GGUF*. Hugging Face model repository, 2026.
- [2] Qwen Team. *Qwen3.8-27B: Hybrid Linear-Attention Dense Model*. Technical report, 2026.
- [3] z-lab. *DFlash 2: Block-Diffusion Speculative Decoding*. GitHub / Hugging Face, 2026.
- [4] KLIK team. *Losing the Draft, Keeping the Speed: Block-Diffusion Speculative Decoding and Prefix-Cached Serving for Long-Context Agentic LLM Workloads on Apple Silicon*. KLIK Research systems report, August 2026. <https://github.com/chengyixu/qwen38-dflash2-bench>.
- [5] Anonymous. *FreeToken: State Reuse Governs Agentic Latency on Unified Memory*. arXiv:2608.16157, 2026.
- [6] Gerganov, G. et al. *llama.cpp*. <https://github.com/ggml-org/llama.cpp>.
- [7] pi contributors. *pi: AI coding assistant*. <https://github.com/badlogic/pi-mono>.